

The Bounded MPMC Queue

Deriving Vyukov's Lock-Free Multi-Producer Multi-Consumer Queue From the Single Conflict That Forces Every Design Decision

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract. The bounded multi-producer multi-consumer (MPMC) queue is the capstone of lock-free queue design. This report does not present Vyukov's algorithm as a finished artifact to be memorized; it reconstructs it as a chain of forced moves, each provoked by the previous one. The single root cause is identified first: to obtain parallelism, the act of *claiming* a queue position must be decoupled from the act of *writing* the data, which means the tail counter advances before the payload is written — the tail “lies.” Every subsequent feature exists to repair the consequences of that lie. We show why a global head/tail pair cannot tell a consumer whether a particular slot is ready; why a per-slot version token is the minimal repair; why that token must encode both the round-trip cycle and the read/write phase in a single monotone value; why the producer cannot use CAS on the slot token (CAS pollutes the very ticket it must preserve) and must instead load-spin; and why the acquire must sit on the loop-exit load, not the pre-loop load. The final code is verified by a multi-threaded harness. Throughout, the emphasis is on the tension that makes each decision inevitable, because understanding why the queue must look the way it does is worth more than reproducing the way it looks.

1. The Root Cause: Claiming and Writing Must Be Decoupled

In the single-producer ring buffer the producer does two things in program order: it writes the slot, then it advances the tail. Because one thread does both in sequence, the advance of the tail necessarily follows the completion of the write. The tail therefore carries an honest meaning — “everything below me is fully written” — and a consumer that observes the tail can trust it.

MPMC shatters this. With many producers, the position each producer will write must be allocated atomically and quickly, by a single atomic instruction (a compare-and-swap or fetch-add on the tail). The allocation cannot wait for the data write — if it did, the writes would serialize and we would lose the very parallelism we built MPMC for. So the order is inverted: a producer first claims a ticket (advancing the tail), and only afterward writes its payload.

The lie of the tail (the single root cause). Because claiming the ticket is decoupled from writing the payload, the tail advances *before* the corresponding data is written. The tail no longer means “the data below me is ready”; it means only “these positions have been handed out.” A consumer that trusts the tail can read a slot whose producer has claimed its ticket but not yet finished — or not even started — writing. Every remaining feature of the MPMC queue exists to compensate for this single lie.

This is the crucial sentence to state *first*. A presentation that opens with “each slot carries a version number” teaches the cure before the disease, and the reader memorizes the token as an incantation. The token is not a starting axiom; it is the forced consequence of decoupling claim from write. Begin anywhere else and the design looks arbitrary.

2. Three Conflicts on a Reused Slot

Strip away every mechanism and ask what can go wrong when two threads touch the same physical slot `buf[i]` of a circular array. Because slot `i` is reused every `N` tickets (positions `i`, `i+N`, `i+2N`, ... all map to it), exactly three conflicts exist.

- **Conflict A (write-write):** two producers try to write slot `i` at the same time.
- **Conflict B (read-write):** a consumer reads slot `i` while a producer is mid-write — a torn read.
- **Conflict C (overwrite-unread):** a producer wants to write slot `i` for a new cycle, but the previous cycle’s value has not yet been consumed — overwriting live data.

All correctness reduces to blocking these three. We now watch which tool blocks which conflict, and which conflicts survive a partial solution — because the survivors are what force the next mechanism.

2.1 A unique ticket kills Conflict A — and nothing else

Give every enqueue a unique, monotonically increasing ticket `t` (producers CAS the `tail_` counter; only one wins each value, the losers retry with a refreshed `t`). The slot is `t & (N-1)`. This immediately removes Conflict A: two producers never hold the same ticket, so they never target the same slot in the same cycle.

But tickets `t` and `t+N` map to the *same* slot one cycle apart. Ticket uniqueness says nothing about cycles, so **Conflicts B and C survive**. The deeper truth: a ticket tells you *who* may write, but a reused physical slot still does not know *which cycle it is currently on* — is it empty and awaiting a write, or full and awaiting a read? The global `head_`/`tail_` counters cannot answer this for an *individual* slot; they track allocation, not per-slot readiness. This is exactly the lie of Section 1, now made concrete.

3. The Per-Slot Token: Forced, Not Invented

Conflicts B and C both come down to one missing fact: *for this physical slot, right now, is it the writer’s turn or the reader’s turn, and for which cycle?* The global counters cannot

supply it. So the information must live *on the slot itself*. We need, per slot, a marker answering two questions: (C) has the previous cycle's value been read, so I may overwrite? and (B) has this cycle's write completed, so I may read?

The minimal encoding is the elegant part. We do not need two fields ("cycle" plus "phase"). A single monotone value — call it the slot's token — encodes both, because the token is set to **the next global ticket entitled to operate on this slot**. Initialise `slots[i].token = i`. Then:

- A producer holding ticket t may write slot $i = t \ \& \ (N-1)$ **only when** `token == t`. Having written, it publishes by setting `token = t + 1`.
- A consumer holding ticket h may read slot $i = h \ \& \ (N-1)$ **only when** `token == h + 1`. Having read, it releases the slot for the *next* cycle by setting `token = h + N`.

The single trick that makes this work: **one value serves as both the read guard and the write guard, separated by a difference of one**. `token == t` means "writer t 's turn"; `token == t+1` means "reader t 's turn"; `token == t+N` means "writer of the next cycle's turn." The token advances monotonically — write adds 1, read adds $N-1$, so a full write-then-read cycle adds exactly N and lands on the next ticket that maps to this slot. The token is a tiny monotone clock private to each slot; producers and consumers consult it to learn whether this instant is their turn.

Why the numbers must be exactly $+1$ and $+N$, and nothing else, is worth dwelling on, because this is where the design is most easily mis-set. The write-side publish must move the token by the smallest amount that flips it from "writer's turn" to "reader's turn": $+1$, giving $t+1$, which is exactly the value the reader holding ticket t waits for. The read-side release must move the token to the *next ticket that maps to this same slot* — and tickets that share a slot are exactly N apart (t and $t+N$ both mask to the same index). So the reader sets $t+N$, i.e. it adds $N-1$ on top of the writer's $+1$. The two increments are forced by their meanings, not chosen: $+1$ is "the minimal flip to the reader," $+N$ is "hand off to the next cycle's writer." Get either wrong and a slot either never becomes readable or never becomes writable again.

The initial value is the same lesson in miniature, and worth recording as a wall the derivation actually hits. The instinctive choice is to initialise every slot's token to 0 (or to leave it zero-filled). That breaks immediately: ticket 1's producer wants slot 1 and waits for `token == 1`, but a zero-initialised slot 1 reads 0, so the first writer of every slot except slot 0 is stuck forever. The token must be primed to "the first ticket that maps here," which is the index itself: `slots[i].token = i`. Only then does the first cycle's writer of slot i (ticket i) find `token == i` and proceed. The zero-init instinct is a genuine trap one walks into and out of; the correct seed i is dictated by the same rule as $+1$ and $+N$ — the token always names the next ticket entitled to the slot, and at time zero that ticket is i .

Why this is the foundation of the whole family. Nothing in this token machine refers to how many producers or consumers there are. It assumes only that tickets are uniquely allocated. SPSC, MPSC, SPMC and MPMC share this exact per-slot state machine; they differ only in whether the global `tail_`/`head_` is advanced by a plain increment (single side) or a CAS (multi side). MPMC is simply the case where both sides CAS. Prove the token machine once and the slot-level correctness of all four queues is proven. And because tickets are monotone integers, not reused pointers, the design is immune to ABA for free — the array version needs no hazard pointers, unlike a linked-list queue.

4. The Producer Path, Derived Step by Step

We now assemble enqueue, and the order in which the pieces fall into place is itself the lesson.

4.1 Claim before you write

A first instinct — load the tail, write the data, then CAS the tail — is wrong, and wrong for a reason worth internalising. Before the CAS succeeds you do not know which ticket you hold, hence which slot is yours; two producers that both read $t = 0$ would both write `slot[0]` before either CAS resolved — Conflict A, reintroduced. The claim must therefore come first: **CAS the ticket, then write the slot the confirmed ticket names.**

4.2 Fullness must be re-checked on every retry

The claim loop must also test fullness, and the test must travel with the retry. Checking $t - h < N$ once before the loop is insufficient: a producer that retries after a failed CAS receives a refreshed t , and the stale pre-loop h may now indicate a full queue. The fullness guard belongs inside the CAS condition so that each attempt re-evaluates it.

One subtlety surfaced during testing and is worth recording honestly. When the loop exits because the queue looked full, the h used was the one loaded before the loop. Since `head_` only advances, that h is a lower bound on the true head, so the queue may in fact no longer be full. Declaring failure on the stale h is overly conservative and, under sustained load, drops items that had room. The fix is to re-load `head_` once before returning false, and only then conclude the queue is genuinely full. This mirrors the SPSC lesson that a fullness test must compare against the freshest available value of the opposite counter.

4.3 On the slot token, CAS is the wrong instrument

Having claimed ticket t , the producer must wait until `slots[loc].token == t` before writing. The instinct is to CAS the token. This is a trap. A failing `compare_exchange` writes the slot's current value back into its expected-value argument — but our expected value is the ticket t , which we must keep *unchanged* (we still need it to address the slot and to publish $t+1$). A CAS would corrupt t . Worse, the semantics we want are not “swap if equal”; they are “wait until equal, then write data, then set $t+1$ ” — a publish that must happen *after* the data write, not atomically with the comparison. So the correct construct is a **load-spin loop**: repeatedly load the token until it equals t , leaving t untouched, then write, then store $t+1$.

The spin should be polite: a `_mm_pause()` in the wait body reduces power draw, relaxes the memory-order pipeline penalty, and yields to a hyperthread sibling. It is not a sleep — no syscall, no descheduling — just a few cycles of courteous idling.

4.4 The acquire must sit on the loop-exit load

The final subtlety, and the one most often missed. The load-spin must establish a happens-before edge so that the payload write (producer) or payload read (consumer) is correctly ordered against the token that gates it. It is tempting to put the acquire only on

the *first* token load before the loop. But if the first load does not yet see the target value, the loop is entered, and the load that actually *releases* the thread — the one whose value finally satisfies the condition — is a load *inside* the loop. If that loop-body load is relaxed, the subsequent payload access may float above it, defeating the guard. Therefore the acquire must be on the loop-body load too (or, equivalently, an acquire fence after the loop). The ordering binds to the load that genuinely lets the thread proceed, not merely to the first probe.

4.5 The assembled producer

```
bool enqueue(const T& item) {
    uint64_t h = head_.load(memory_order_relaxed);
    uint64_t t = tail_.load(memory_order_acquire);
    while (t - h < N &&
           !tail_.compare_exchange_weak(t, t + 1,
                                         memory_order_release, memory_order_relaxed)) { }
    if (t - h >= N) { // re-check against freshest head
        h = head_.load(memory_order_relaxed);
        if (t - h >= N) return false; // genuinely full
        while (t - h < N &&
               !tail_.compare_exchange_weak(t, t + 1,
                                             memory_order_release, memory_order_relaxed)) { }
        if (t - h >= N) return false;
    }
    uint64_t loc = t & (N - 1);
    uint64_t tok = slots[loc].token.load(memory_order_acquire);
    while (tok != t) { // load-spin (NOT cas: cas would clobber t)
        _mm_pause();
        tok = slots[loc].token.load(memory_order_acquire); // acquire on the releasing load
    }
    slots[loc].data = item; // write AFTER token==t
    slots[loc].token.store(t + 1, memory_order_release); // publish: now readable
    return true;
}
```

5. The Consumer Path: The Exact Dual

The consumer mirrors the producer with the roles of `head_` and `tail_` swapped and the token comparisons shifted by one. It claims a read ticket `h` by CAS on `head_`; the queue is empty when `h == t` (the read ticket has caught the write ticket). It then load-spins until `token == h + 1` — the writer's published value — reads the payload, and releases the slot for the next cycle with `token = h + N`. The same two rules apply: write the payload access on the correct side of the token, and put the acquire on the loop-exit load.

```

bool dequeue(T& out) {
    uint64_t t = tail_.load(memory_order_relaxed);
    uint64_t h = head_.load(memory_order_acquire);
    while (h < t &&
           !head_.compare_exchange_weak(h, h + 1,
                                         memory_order_release, memory_order_relaxed)) { }
    if (h >= t) { // re-check against freshest tail
        t = tail_.load(memory_order_relaxed);
        if (h >= t) return false; // genuinely empty
        while (h < t &&
               !head_.compare_exchange_weak(h, h + 1,
                                             memory_order_release, memory_order_relaxed)) { }
        if (h >= t) return false;
    }
    uint64_t loc = h & (N - 1);
    uint64_t tok = slots_[loc].token.load(memory_order_acquire);
    while (tok != h + 1) { // wait for writer's publish
        _mm_pause();
        tok = slots_[loc].token.load(memory_order_acquire);
    }
    out = slots_[loc].data; // read BEFORE releasing
    slots_[loc].token.store(h + N, memory_order_release); // release for next cycle
    return true;
}

```

6. Verification

A multi-threaded harness exercises the queue with four producers and four consumers on a capacity-1024 ring. Each producer emits a disjoint block of distinct integers, so the full set of produced values is exactly the interval $[0, \text{TOTAL})$; each consumer records every value it dequeues into an atomic seen-array, aborting immediately on any duplicate. After the producers join and the consumers drain, the harness verifies that no value is missing and none is duplicated.

Result. Across repeated runs of one million items ($4 \text{ producers} \times 250,000$), the harness reports produced = consumed = 1,000,000, missing = 0, duplicated = 0 — no loss, no duplication, no torn or garbage values. The conservative-fullness fix of Section 4.2 was required: without re-loading the opposite counter before declaring full or empty, the harness loses items under sustained load. The fix is precisely the SPSC lesson that fullness/emptiness must be judged against the freshest value of the opposite counter.

7. Relation to the Original Vyukov Algorithm

The queue derived here is, in essence, Vyukov’s 2010 bounded MPMC queue, reached from the conflict analysis rather than copied. It is worth being precise about what coincides and where the two presentations diverge, because the divergence explains a wrinkle the verification exposed.

What coincides exactly. The per-slot token machine is identical. Vyukov initialises each cell’s sequence to its index ($\text{seq}[i] = i$); the producer publishes with $\text{seq} = \text{pos} + 1$; the

consumer releases with $\text{seq} = \text{pos} + N$. These are precisely the $+1 / +N$ and the seed i derived above — arrived at independently by the “what flips the guard / what hands off the cycle” argument, and identical to the original. The slot-level state machine is the same algorithm.

Where the two diverge: how fullness and emptiness are judged. Vyukov never reads the opposite counter. His producer reads the cell’s own sequence and computes $\text{dif} = \text{seq} - \text{pos}$: $\text{dif} == 0$ means “my turn, CAS the position”; $\text{dif} < 0$ means “full, return false”; $\text{dif} > 0$ means “someone beat me, reload pos.” Fullness, turn-arbitration, and retry are folded into a single comparison on the cell, and the producer never touches `dequeue_pos`. The version derived here instead judges fullness by the global difference $\text{tail}_- - \text{head}_- < N$ (and emptiness by $\text{head}_- < \text{tail}_-$), reading the opposite counter. This is the direct cause of the conservative-fullness wrinkle of Section 4.2: because the opposite counter read can be stale, it must be re-read before declaring full or empty — a step Vyukov’s cell-local test never needs. Two valid routes to the same guarantee: the global-counter test is more intuitive; the cell-local test is more economical (one comparison, no opposite-counter traffic, exactly one CAS per operation).

A second, smaller divergence: Vyukov uses a single `for(;;)` that checks readiness, claims the ticket, and exits in one CAS; the derivation here splits this into two loops — first CAS the ticket, then load-spin the slot token — which is easier to reason about step by step but structurally one layer larger.

On equivalence. The two are the same algorithm under two presentations. Equivalence can be shown from (a) the identity of the per-slot token machine — token values, increments, and seed all coincide — and (b) the equivalence of the two fullness/emptiness predicates (both decide the single question “is the cell for this position, on this cycle, ready for the operation?”, one reading it locally from the cell, the other globally from the counters). A formal linearizability proof can be built on those two facts; we omit it deliberately, because the value of this report is the chain of forced design moves, not a formal cross-check of two equivalent forms.

8. Synthesis

Every feature of the bounded MPMC queue traces back to a single decision made for the sake of parallelism: claiming a position is decoupled from writing it, so the tail advances before the data lands and therefore lies. From that lie, the rest is forced. The global counters cannot certify an individual slot, so readiness information descends onto each slot as a token. One monotone token suffices because it names the next ticket entitled to the slot, encoding cycle and phase together with the read guard and write guard a single step apart. CAS is the wrong instrument on that token because a failed CAS would corrupt the very ticket we must preserve, so a polite load-spin takes its place. And the acquire must bind to the load that actually releases the thread, not the first probe. None of this is arbitrary; each step is the unique repair for the damage the previous step left behind. That is why the queue is worth deriving rather than copying — and why the same token machine, once

understood here, gives SPSC, MPSC, and SPMC as nothing more than special cases of how the two global tickets are advanced.